

Placement Empowerment Program
Cloud Computing and DevOps Centre

Set Up Git Branching

**Create a new branch in your Git repository for testing.
Add a new feature and merge it.**

Name: Neithal Pillai

Department : IT

Introduction and Overview

This Proof of Concept (POC) demonstrates the process of creating and managing branches in Git, adding a new feature to a project, and merging that feature into the main branch. Git branching is a fundamental practice in version control that allows developers to work on new features or changes without affecting the main project. By completing this POC, you will learn how to work with Git branching, manage feature development, and maintain a clean repository.

Objective

The primary objective of this project is to:

1. **Create a New Branch for Testing** – Learn how to create a new branch in a Git repository to work on a new feature without affecting the main branch.
2. **Add a New Feature** – Implement a simple feature by modifying or adding files to the project.
3. **Merge the Changes** – Merge the newly created branch back into the main branch once the feature has been tested.

Importance of Git Branching

Git branching plays a key role in modern software development by enabling parallel development of features and bug fixes without disrupting the main codebase. The key advantages of using Git branching include:

- **Isolation of Features** – Working in a separate branch ensures that new features or bug fixes do not interfere with the main project until they are ready.
- **Ease of Collaboration** – Developers can work on different branches simultaneously, which helps avoid conflicts and ensures smooth collaboration in team environments.
- **Efficient Workflow** – Git branching allows for a more organized workflow, making it easier to test and refine features without impacting the live project.

By completing this POC, you will gain hands-on experience in creating and managing branches, adding new features, and merging them back into the main branch using Git. These skills are essential for collaborative software development and will help you maintain a clean, organized codebase.

Steps for execution:

Step 1: Open Git Bash

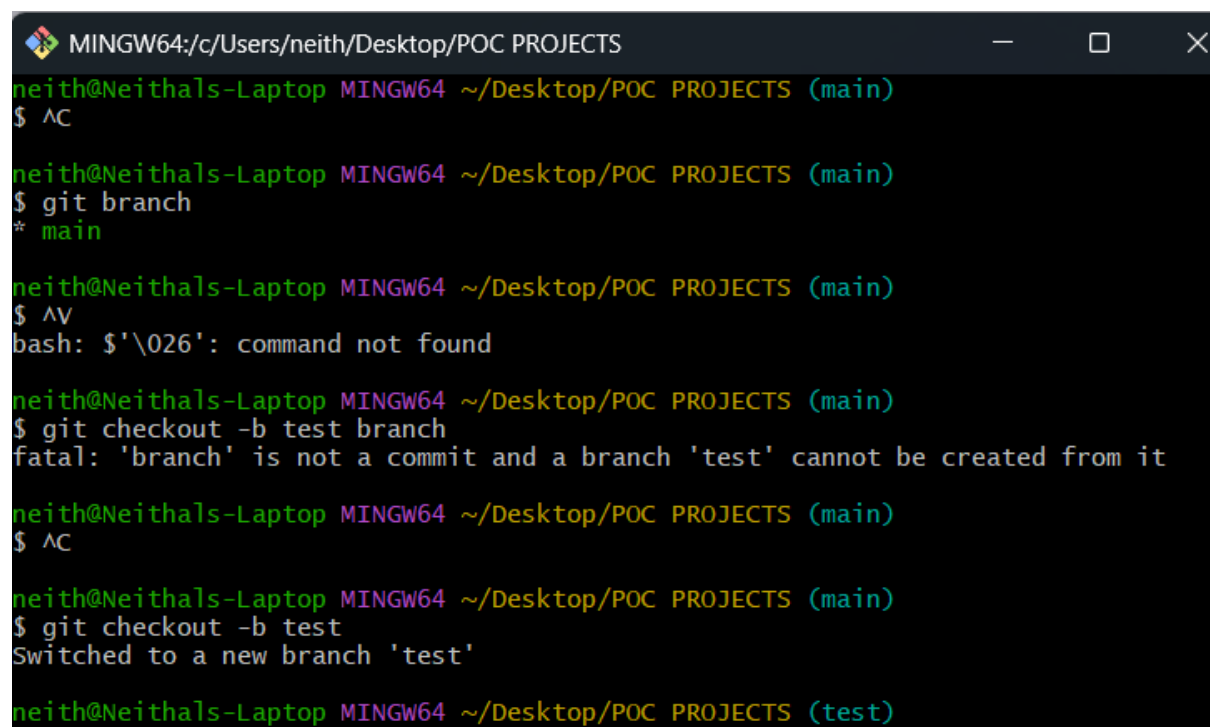
I opened **Git Bash** from the Start Menu to begin working with Git on my Windows machine.

Step 2: Navigate to My Repository

Using the `cd` command, I navigated to the folder where my Git repository was located. This allowed me to work on the project files.

Step 3: Check Current Branch

Before making any changes, I checked which branch I was currently on by running the `git branch` command. This helped me confirm that I was working on the correct branch.

A screenshot of a Git Bash terminal window. The title bar shows the path "MINGW64:/c/Users/neith/Desktop/POC PROJECTS". The terminal content shows the user "neith@Neithals-Laptop" in the "MINGW64" environment at the directory "~/Desktop/POC PROJECTS" on the "main" branch. The user enters the command `$ ^C` (which appears as `$ ^C` in the image). Then, they enter `$ git branch`, and the output shows `* main`. Next, they enter `$ ^V` (which appears as `$ ^V` in the image), and the terminal shows an error: `bash: $'\026': command not found`. Then, they enter `$ git checkout -b test branch`, and the terminal shows a fatal error: `fatal: 'branch' is not a commit and a branch 'test' cannot be created from it`. Finally, they enter `$ ^C`, then `$ git checkout -b test`, and the terminal shows `Switched to a new branch 'test'`. The prompt now shows the user is on the "test" branch.

Step 4: Create a New Branch

To add a new feature without affecting the main branch, I created a new branch using the `git checkout -b` command. This both created and switched me to the new branch automatically.

```

neith@Neithals-Laptop MINGW64 ~/Desktop/POC PROJECTS (main)
$ git checkout -b test
Switched to a new branch 'test'

neith@Neithals-Laptop MINGW64 ~/Desktop/POC PROJECTS (test)
$ git add .

neith@Neithals-Laptop MINGW64 ~/Desktop/POC PROJECTS (test)
$ git commit -m "Added a new html file to demonstrate branch testing"
[test 2bfa475] Added a new html file to demonstrate branch testing
8 files changed, 150 insertions(+), 2 deletions(-)
create mode 100644 @echo off.bat
create mode 100644 Mywebsite.html
create mode 100644 Mywebsite.txt
create mode 100644 Slide1.PNG
create mode 100644 Testbranch.html
create mode 100644 backup_git.bat

```

Step 5: Add a New Feature

I then made the necessary changes to my project, whether it was modifying existing files or adding new ones to implement the feature I wanted to test.

Step 6: Stage and Commit the Changes

After completing my changes, I staged them using the git add command, which prepared the files for commit. I then committed the changes with a descriptive message to keep track of the new feature.

```

neith@Neithals-Laptop MINGW64 ~/Desktop/POC PROJECTS (main)
$ git merge test
Updating 410755e..2bfa475
Fast-forward
 @echo off.bat | 3 +++
 Mywebsite.html | 51 +++++
 Mywebsite.txt | 51 +++++
 README.md | 3 +-
 Slide1.PNG | Bin 0 -> 138927 bytes
 Testbranch.html | 29 +++++
 backup_git.bat | 13 +++++
 index.html | 2 +-
8 files changed, 150 insertions(+), 2 deletions(-)
create mode 100644 @echo off.bat
create mode 100644 Mywebsite.html
create mode 100644 Mywebsite.txt
create mode 100644 Slide1.PNG
create mode 100644 Testbranch.html
create mode 100644 backup_git.bat

```

Step 7: Switch Back to the Main Branch

Once my work on the new feature was complete, I switched back to the **main branch** to prepare for merging the changes.

```

neith@Neithals-Laptop MINGW64 ~/Desktop/POC PROJECTS (main)
$ git push origin main --force
Enumerating objects: 15, done.
Counting objects: 100% (15/15), done.
Delta compression using up to 12 threads
Compressing objects: 100% (13/13), done.
Writing objects: 100% (15/15), 125.35 KiB | 15.67 MiB/s, done.
Total 15 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/Neithal07/POC-Projects.git
+ ab8d50b...2bfa475 main -> main (forced update)

neith@Neithals-Laptop MINGW64 ~/Desktop/POC PROJECTS (main)
$ git commit
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

neith@Neithals-Laptop MINGW64 ~/Desktop/POC PROJECTS (main)
$ |

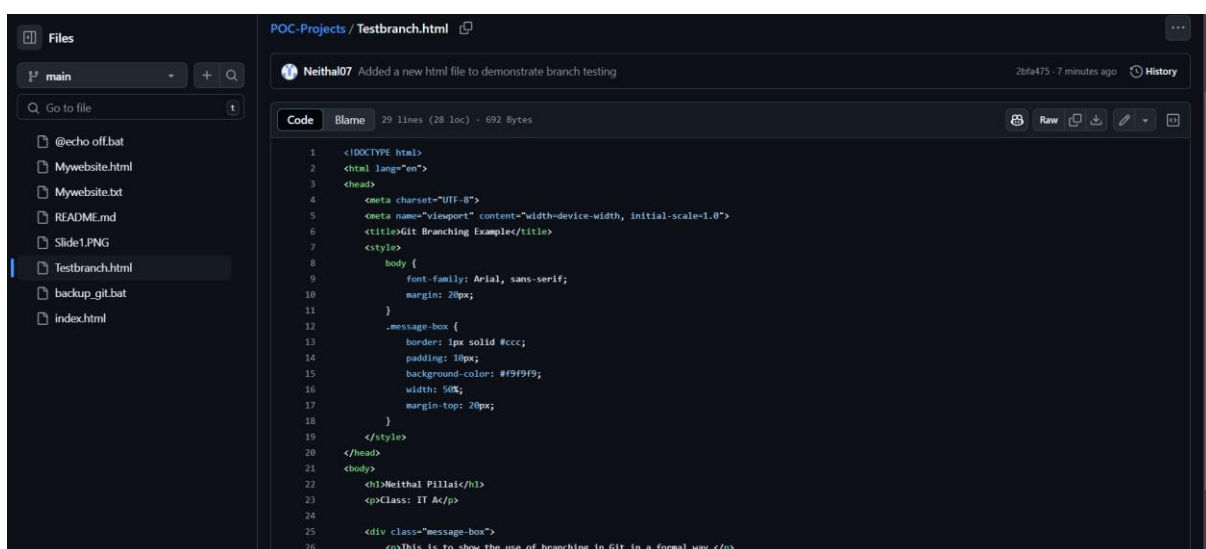
```

Step 8: Merge the Testing Branch into Main

On the main branch, I merged the changes from my testing branch using the git merge command. This incorporated the new feature into the main branch.

Step 9: Resolve Any Merge Conflicts

If there were any merge conflicts, I manually resolved them by editing the affected files. After resolving the conflicts, I staged and committed the changes to complete the merge.



```

POC-Projects / Testbranch.html
Neithal07 Added a new html file to demonstrate branch testing 2bfa475 · 7 minutes ago History

Code Blame 29 lines (28 loc) · 692 Bytes
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Git Branching Example</title>
7   <style>
8     body {
9       font-family: Arial, sans-serif;
10      margin: 20px;
11    }
12    .message-box {
13      border: 1px solid #ccc;
14      padding: 10px;
15      background-color: #f9f9f9;
16      width: 50%;
17      margin-top: 20px;
18    }
19  </style>
20 </head>
21 <body>
22   <h1>Neithal Pillai</h1>
23   <p>Class: IT Ac</p>
24
25   <div class="message-box">
26     <p>This is to show the use of branching in Git in a formal way.</p>

```

Key Takeaways

Upon completion of this Proof of Concept (POC), you will be able to:

1. Create and Manage Git Branches

Learn how to create, switch between, and manage branches in a Git repository, enabling isolated development for new features or bug fixes.

2. Implement and Test New Features

Gain the ability to add and modify features in a separate branch, allowing you to test changes without affecting the main project.

3. Merge Changes and Resolve Conflicts

Understand how to merge feature branches back into the main branch and resolve any merge conflicts that may arise.

4. Maintain an Organized and Clean Repository

Develop skills in keeping your Git repository clean by deleting unnecessary branches and ensuring the repository remains manageable for future development.